

Foundational Skills Program

Course Title: Software Testing



Date: May 10, 2026

Md. Golam Mostofa

AI/ML-QA Engineer, Bizzntek Ltd



Outline

- Introducing Evaluation dataset
- Introducing DeepEval and Uptrain tool
- Evaluation Setup
- Environment Setup
- Test EDEG chatbot using deepEval or Uptrain (automated and AI-Driven)

What Are Evaluation Datasets?

Definition: Curated collections of questions, prompts, and expected answers used to measure LLM capabilities across dimensions like knowledge, reasoning, truthfulness, and math.



Standardized

Fixed question sets with ground-truth answers enable fair, reproducible comparisons across models and versions.



Multi-Dimensional

Cover diverse skills: factual recall, logical reasoning, coding, mathematics, common sense, and safety alignment.



Benchmarking

Enable tracking model progress over time and comparing proprietary vs. open-source models objectively.

MMLU

Massive Multitask Language Understanding

57

Subjects

15,908

Questions

4-way

MCQ Format

STEM+

Coverage

Categories



STEM: Math, Physics, Chemistry, CS



Humanities: History, Law, Philosophy



Social Sciences: Economics, Psychology



Medical: Anatomy, Clinical Knowledge



Professional: Law, Accounting, Finance

Why Use MMLU?

- Gold-standard breadth benchmark for general knowledge
- Widely adopted; enables cross-paper comparison
- Reveals gaps across professional & academic domains
- Hard enough to differentiate strong vs. weak models

Measuring Truthfulness in Language Models

Problem it Solves: LLMs often generate plausible-sounding but false answers — especially for misconceptions, myths, and conspiracy-prone topics. TruthfulQA directly targets these failure modes.

817 Questions

Hand-crafted adversarial prompts

Questions designed so that humans would answer correctly but LLMs trained on text tend to fail due to imitative falsehoods.

38 Categories

Misconceptions & myths

Covers health, finance, law, conspiracy, fiction, politics — wherever false beliefs are commonly held.

Dual Metrics

Truthful & Informative

Scores both whether an answer is true AND whether it's useful — a trivially safe 'I don't know' is penalised.

Overview

~37,000

annotated math word problems

5-way MCQ

with detailed rationales

Derived from

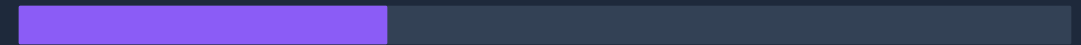
AQuA-RAT dataset

Tests

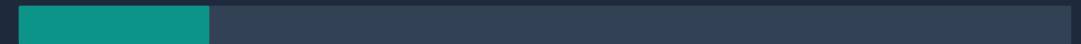
multi-step arithmetic reasoning

Problem Categories

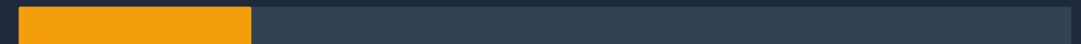
Arithmetic & Algebra **35%**



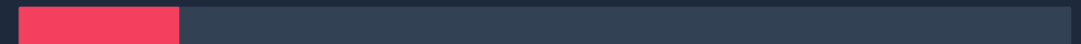
Geometry **18%**



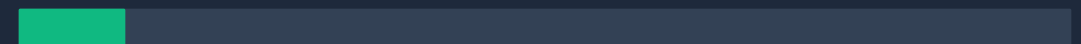
Probability & Statistics **22%**



Physics Word Problems **15%**



Other Applied Math **10%**



Other Notable Evaluation Datasets

HellaSwag

Commonsense

70k sentence completion tasks testing situational commonsense reasoning. Uses adversarial filtering to make it challenging.

GSM8K

Grade-School Math

8.5k linguistically diverse grade-school math problems requiring multi-step arithmetic. Key test for chain-of-thought reasoning.

HumanEval

Code Generation

164 hand-crafted Python programming problems. Tests functional correctness: code must pass hidden unit tests.

BIG-Bench

Diverse Tasks

Beyond the Imitation Game: 204 tasks from 132 institutions testing capabilities beyond standard benchmarks.

ARC

Science QA

AI2 Reasoning Challenge: 7,787 science questions split into Easy and Challenge sets for deeper reasoning.

WinoGrande

Pronoun Resolution

44k Winograd schema problems testing commonsense reasoning about pronoun references at scale.

DeepEval — Usage & Features

Python

```
from deepeval import evaluate
from deepeval.metrics import AnswerRelevancyMetric
from deepeval.test_case import LLMTestCase

# Define the metric
metric = AnswerRelevancyMetric(threshold=0.7)

# Create test case
test_case = LLMTestCase(
    input=
    "What is LangChain?"

    actual_output=llm_response
)

# Run evaluation
evaluate([test_case], [metric])
```

Key Features

pytest Integration

Write evals like unit tests; run with deepeval test run

Confident AI Dashboard

Visual UI to track regressions & compare runs

CI/CD Ready

Fail pipelines when metric scores drop below threshold

LLM-as-Judge

Uses GPT-4 or custom models to score open-ended answers

RAG Support

Full pipeline metrics: retrieval, faithfulness, recall

Core Philosophy: UpTrain focuses on production-grade LLM observability, evaluating, monitoring, and improving AI pipelines continuously, not just at test time.

Response Quality

Scores completeness, conciseness, and relevance of generated text.

Context Grounding

Checks if responses are grounded in provided context (critical for RAG).

Conversation Eval

Multi-turn conversation quality: coherence, satisfaction, topic drift.

Drift Detection

Monitors data drift and model degradation in production over time.

Safety & Bias

Detects harmful content, PII leakage, and demographic bias.

Custom Operators

Write custom Python operators to embed domain-specific evaluation logic.

UpTrain — Usage & Dashboard

Python

```
import uptrain

# Initialize the evaluator
eval_llm = uptrain.EvalLLM(uptrain.Evals.RAG_WITH_CITATION,
    openai_api_key="sk-..."
)

# Evaluate responses
results = eval_llm.evaluate(
    data=[{
        "question": "What is RAG?",
        "context": retrieved_docs,
        "response": llm_answer
    }]
)

print(results)
```

Platform Features

Self-Hosted Dashboard

Local web UI — no data leaves your infra

UpTrain Cloud

Managed SaaS with experiment tracking

Slack Alerts

Real-time alerts when quality drops in prod

A/B Testing

Compare prompt versions with statistical rigor

LangChain/LlamaIndex

Native integrations with popular LLM frameworks

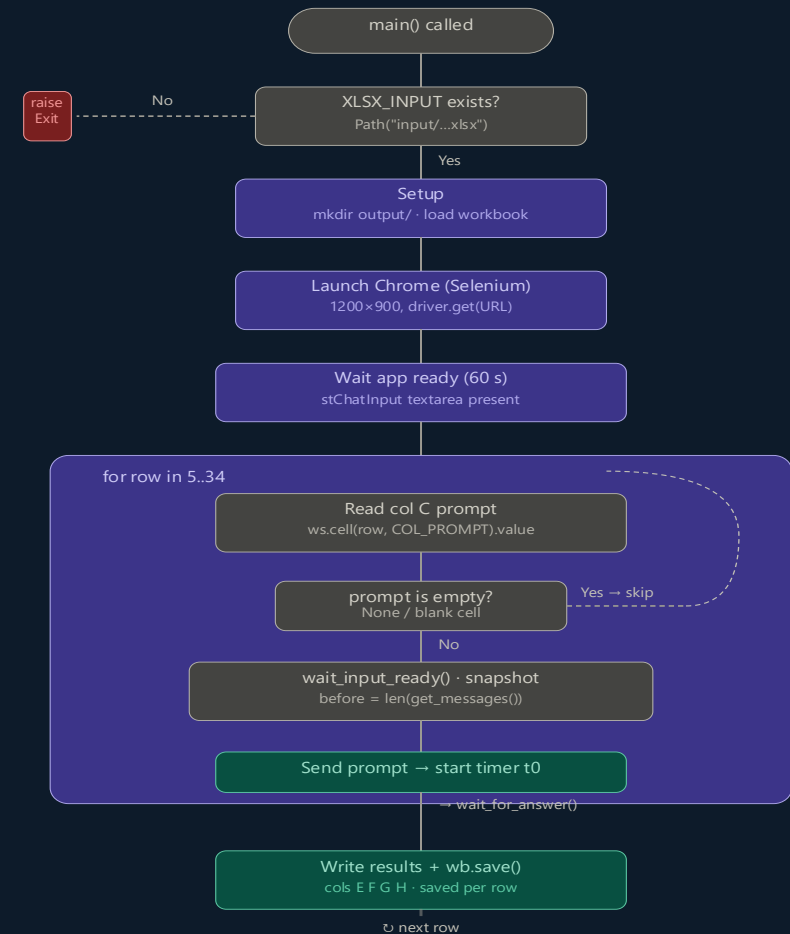
DeepEval vs UpTrain

Dimension	DeepEval	UpTrain
Primary Focus	Unit-test-style LLM evaluation	Production monitoring & observability
Integration	pytest, CI/CD pipelines	LangChain, LlamaIndex, REST API
Metrics Style	Pre-built + custom via G-Eval	Pre-built operators + custom Python
Dashboard	Confident AI (cloud)	Self-hosted or UpTrain Cloud
RAG Support	Full RAG metric suite 	Full RAG metric suite 
Alerting	Threshold-based test failure	Real-time Slack/email alerts
Best For	Dev/testing phase evaluation	Production quality monitoring



EDEG Chatbot Test automation Architecture

- Start from main function
- Checking test file available or not
- Result folder checking
- Launch app in Chrome browser
- Wait for ready
- Put prompt into the chatbot
- Get response
- Test response by the Mistral LLM
- Same test result



Pre-Requisites

```
root@ae68fead5916:/# ollama list
```

NAME	ID	SIZE	MODIFIED
llama3.2:1b	baf6a787fdff	1.3 GB	29 seconds ago
nomic-embed-text:latest	0a109f422b47	274 MB	About a minute ago

```
root@ae68fead5916:/#
```

Key Features



Ollama install



Chrome-browser



Llama3.2:1b



Nomic-embed-text

Environment Setup For EDEG Chatbot



1 Create Virtual Environment & Active

```
python -m venv venv  
venv\Scripts\activate
```

Download the [EDEG Chatbot](#) first

2 Install Requirements

```
pip install -r requirements.txt
```

Install the Python packages.

3 Run the Chatbot

```
streamlit run app.py
```

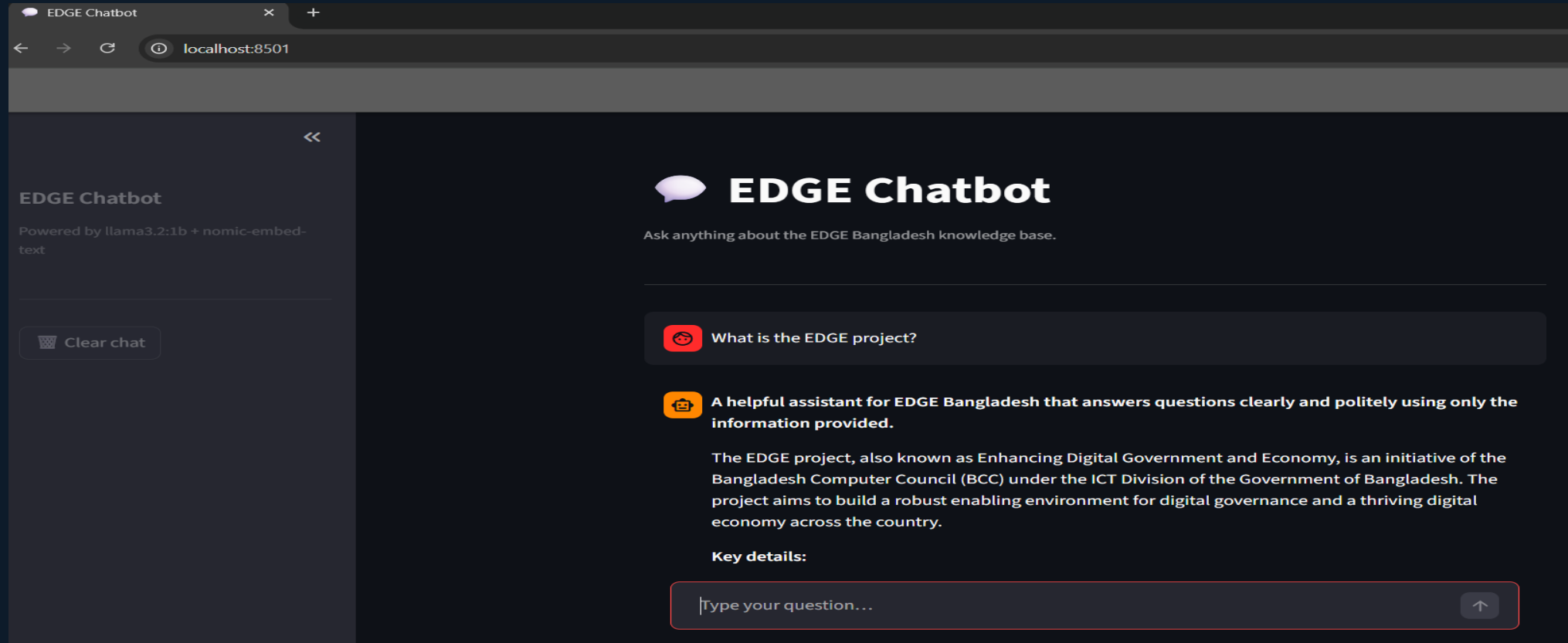
Navigate: <http://localhost:8501/>

4 Test the Chatbot

```
venv\Scripts\activate  
python fill_test_sheet.py
```

Run the test.

Test EDGE Chatbot Automatically



Q&A

Any Questions?



Thanks



MD. GOLAM MOSTOFA



golam.mostofa@bizzntek.com